

Assignment 1 advance

part 1:

1.MainApp.java

It performs the following:

```
ConnectionManager connManager = new
```

```
ConnectionManager(); // It creates a simple connection
```

manager whose job is to create a new connection on demand.

```
TemplateManager templateManager = new
```

```
TemplateManager();// This manager builds job templates,
```

with a heavy (sleep 3 seconds) process to simulate the creation of a realistic template.

```
JobExecutor executor = new JobExecutor(connManager);//
```

An object responsible for executing functions according to their type (EMAIL/DATA/REPORT).

It uses ConnectionManager to establish a connection during execution.

```
User alice = new User("alice", Arrays.asList("EMAIL",  
"REPORT")); // The user who requests jobs is identified, and  
the type of jobs they are allowed to perform is specified
```

```
Job reportJob =  
templateManager.buildReportJobTemplate("MonthlyReport",  
"format=PDF;brand=TaskMaster").createJobInstance();//
```

A heavy report template is built.

Then a new job is created from this template.

The process takes time (3 seconds) because the template is built from scratch.

Then the user is assigned:

```
reportJob.setRequestedBy(alice);
```

```
executor.executeJob(reportJob); //
```

JobExecutor performs the following functions:

- Print job information

- Obtain a connection

- Select the correct execution method based on the job type

- Execute queries

- Close the connection

Job reportJob2 =

templateManager.buildEmailJobTemplate("Monthly email
Report", "format=PDF;all=true").createJobInstance(); // builds
from scratch

reportJob2.setRequestedBy(alice);

System.out.println("\n--- Execute job (naive executor) ---");

```
executor.executeJob(reportJob2);//
```

The following steps are performed:

- Building a new email template from scratch
- Creating a new job
- Assigning the user
- Executing the job using JobExecutor

2.Connection.java

This class represents a single database connection in a simplified manner:

- It contains an ID to identify each connection.
- It contains the `executeQuery()` function, which simulates the execution of an SQL query by simply printing it to the screen.

This connection is used to execute functions within JobExecutor.

3.ConnectionManager.java

This class is responsible for:

- Creating a new connection using `createConnection()`.
- Maintaining the number of connections created (counter).
- Closing a connection using `closeConnection()` by simply printing a message.

There is no actual connection management; it's just simple creation and closing.

4.User.java

This class represents a user within the system and contains their basic information.

The class performs the following functions:

- Stores the username.
- Maintains a list of permissions the user is allowed to perform (such as EMAIL or REPORT).

- Provides a `hasPermission()` function to check if the user has permission to run a specific type of function.

This class is used to determine who requested the function and to verify that they are authorized to execute it within the system.

5.Job.java

This class represents a job within the system, specifically the module executed by the `JobExecutor`.

The class performs the following functions:

- Stores the job ID.
- Defines the job type as text (e.g., EMAIL, DATA, REPORT).
- Stores the job name that describes it.
- Retains the job's configuration as text.
- Stores the user who requested the job execution using the `requestedBy` property.

These objects are used to represent tasks performed by the system and are passed to the JobExecutor to complete the execution process.

6.HeavyTemplate.java

This is a template from which functions are built:

- It contains basic information (type, name, config).
- It contains the template body.
- It has a createJobInstance() function to create a new function from this template.

This class is the foundation upon which TemplateManager builds new functions.

7.TemplateManager.java

This class is responsible for:

- Building the three templates:

I. Email Template

II. Data Processing Template

III. Report Template

Each template:

- Calls `simulateHeavyLoad()` to simulate a heavy build that takes 3 seconds.

- Creates a new `HeavyTemplate` each time.

This template is later used to create jobs.

8.JobExecutor.java

This class represents the part responsible for executing jobs within the system.

It is the component that receives the job and executes it step by step.

The class performs the following:

- Obtains a connection from the `ConnectionManager` before starting execution.

- Prints information about the job to be executed.
- Identifies the job type (EMAIL, DATA, or REPORT) by examining the type value.
- Calls the appropriate execution function for each job type:
 - executeEmailJob to execute the email job
 - executeDataJob to process the data
 - executeReportJob to generate reports
- Executes database queries related to each job using the connection.
- Closes the connection after the operation is complete to ensure proper execution.

This class is the primary "engine" of the system, as it handles the execution of any job passed to it and completes all the necessary steps for its execution.

Part2:

The problems:

1.MainApp

-Tight Coupling

The MainApp is directly linked to ConnectionManager, TemplateManager, and JobExecutor as concrete classes.

It's difficult to change the execution (e.g., to ConnectionPool or Proxy) without modifying the code.

-Repetition in Job Creation and Execution Logic

The same steps are repeated:

--Build template

--createJobInstance

--setRequestedBy

--executeJob

This is repeated twice, and adding more jobs will result in even more repetition.

- Using the naive version of everything

- Naive ConnectionManager (without Pool)

- Naive TemplateManager (builds from scratch)

- Naive JobExecutor (if/else)

This contradicts the company's future requirements (Connection Pool, Strategy, Proxy).

2.User

- Representing permissions only as strings (EMAIL, REPORT,etc.) means that any typo (EAMIL) breaks logic without a clear error.

- There is no real permission model (no enum, no role, no permission object). Developing a more robust permission system later (Admin, Manager) is difficult.

- There are limitations in scalability. If we need more complex permission types (create, update, delete jobs), we will need to make many code changes.

3.job

- The job type is stored as a string: EMAIL, DATA, REPORT

It relies on literal correct spelling (case-sensitive and typo-prone).

This violates type safety principles; using (enum JobType) would be preferable.

- The configuration is stored as a single string (serialized string):

```
//format=PDF;all=true
```

This is difficult to read, modify, or verify.

It violates SRP because the Job uses disorganized configuration logic.

- The job is mutable in terms of requestedBy. ,requestedBy can be changed at any time using setRequestedBy().

This can cause confusion regarding who actually requested the job if it is changed after creation.

4.ConnectionManager

The ConnectionManager class suffers from several problems:

- The counter is not static.

This allows an unlimited number of connections if we create more than one ConnectionManager, breaking the)maximum 10 connections(logic.

- There is no real Connection Pool.

Connections are not stored or reused; a new connection is created each time.

- Exceeding the MAX limit returns null

which subsequently causes a NullPointerException.

- The `closeConnection()` does not reconnect the connection pool.

Closing is simply a printout, with no proper resource management.

- There is no thread-safety management.

The counter is not protected, and multithreading issues can occur.

5. HeavyTemplate

- It combines two responsibilities:

It represents a template

And simultaneously builds a job.

This violates the Single Responsibility Principle (SRP).

- It doesn't use a Prototype Pattern.

Every time, a new job is built in a naive way.

There's no professional template clone.

- The way the ID is built is illogical.

It uses(`templateBody + type + timestamp`).

This makes the ID dependent on the template text itself, which is a poor design.

- Type, as a string.

Same problem as a job: it relies on a string, not an enum.

6.TemplateManager

- Excessive code duplication.

The three functions are almost identical with only minor text changes.

The code smells obvious (violates DRY).

- Each template creation takes a long time (simulate HeavyLoad).

We build a new template from scratch every time with

`Thread.sleep(3000)`.

There is no caching or template reuse.

Performance is very poor when creating a large number of jobs.

- There is no registry for managing templates.

The job requests JobTemplateRegistry.

The current class does not store or reuse templates.

- Mixed responsibilities.

Builds the template.

Determines its type (EMAIL/DATA/REPORT). These responsibilities could be better separated into layers later.

7.JobExecutor

- A long if/else string based on a String type.

This violates the Open/Closed Principle (OCP):

- Any new Job type means modifying the class itself.

- We should use a Strategy Pattern instead of if/else.

- Good Class / Too Many Responsibilities

JobExecutor performs:

- Logging (printing start and end)
- Connection management (acquire/close)
- Selecting execution logic based on the job type
- Executing business details (email, data, report)

This reduces cohesion and increases coupling.

-There is no proxy layer.

The assignment requires:

- A proxy to check permissions
- Measure time
- Acquire connections from the pool
- Record logs.

But now everything happens within JobExecutor itself.

-Using ConnectionManager instead of ConnectionPool

Takes connections naively

No reuse, no blocking, and no pool management.

-There is no actual error handling
the code assumes everything works.

No try/catch function within executeEmailJob, executeDataJob, or executeReportJob.

No retry or error logging.

8.Connection

The problems here are simple:

It's just a mock/simulation of a connection, and its main issue isn't the design, but rather that it's too simplistic and doesn't represent a real connection. However, within the context of this exercise, it's acceptable.

Part3:

1. **Connection Pool**

In the original version:

- A new connection was created each time without being reused.
- There was no upper limit to the number of connections.

- There was no blocking behavior when connections were full.
- The ConnectionManager was a simple and insecure object.

Solution using Connection Pool:

The Connection Pool pattern was implemented as follows:

Create the ConnectionPool interface:

```
package connections;

public interface ConnectionPool {

    Connection acquire() throws
    InterruptedException;

    void release(Connection connection);
}
```

Create a SimpleConnectionPool:

```
package connections;
```

```
import
java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.BlockingQueue;

public class SimpleConnectionPool implements
ConnectionPool {

    private final BlockingQueue<Connection>
pool;

    // DEFAULT constructor (fixes the red
error)

    public SimpleConnectionPool() {

        this(10); // default pool size

    }

    // custom constructor

    public SimpleConnectionPool(int maxSize) {

        this.pool = new
ArrayBlockingQueue<>(maxSize);

        for (int i = 1; i <= maxSize; i++) {
```

```

        pool.offer(new Connection("Conn-"
+ i));

    }

}

@Override

public Connection acquire() throws
InterruptedException {

    return pool.take(); // waits if empty

}

@Override

public void release(Connection connection)
{

    if (connection != null) {

        pool.offer(connection);

    }

}

}

```

A real pool is implemented using BlockingQueue:

- It allows a maximum of (10) connections.
- It returns a ready connection on request.
- It waits for (blocks) if a connection is unavailable.

Using the Pool Inside the Proxy

JobExecutor is no longer responsible for establishing connections.

Now:

- The proxy receives the connection from the pool
- Then it passes it to JobExecutor
- And finally it returns it to the pool

2. Prototype Pattern

The problem with the old system:

TemplateManager was rebuilding each template from scratch using:
//Thread.sleep(3000)

This resulted in:

- Extreme slowness
- Unnecessary redundancy
- Lack of scalability

To solve using Prototype Pattern:

Create a JobPrototype interface

```
package templates
```

```
import job.Job;

public interface JobPrototype {
    Job cloneJob();
}
```

Creating real templates for each job type

- EmailJobTemplate

-DataJobTemplate

-ReportJobTemplate

Each template contains:

//name

//config

//type

//method :cloneJob()

Ex:

```
public Job cloneJob() {  
    String id = type + "-" + System.currentTimeMillis();  
    return new Job(id, type, name, config);  
}
```

Building a registry of templates:

```
JobTemplateRegistry registry = new JobTemplateRegistry();
```

Using a clone instead of rebuilding:

```
Job reportJob = registry.get("REPORT").cloneJob();
```

3.Strategy Pattern:

The problem with the old system:

JobExecutor relied on if/else.

This is contrary to the OCP principle.

The solution using Strategy Pattern

Creating a Job Execution Strategy Interface:

```
package executor;  
  
import job.Job;  
  
public interface JobExecutionStrategy {  
    void execute(Job job);  
}
```

Creating independent strategies:

-EmailJobStrategy

-DataJobStrategy

-ReportJobStrategy

Creating a Strategy Factory to Defining the Appropriate Strategy:

```
JobExecutionStrategy strategy = factory.getStrategy(job.getType());
```

JobExecutor update to become:

```
JobExecutionStrategy strategy = factory.getStrategy(job.getType());  
strategy.execute(job);
```

4.Proxy Pattern:

The problem with the old system:

JobExecutor used to do everything:

Execute the job

Manage connections

Verify permissions

Measure time

Print

-This violates SRP and makes the class very complex.

The solution using Proxy Pattern

It performs the following:

1. Verifying user permissions:

```
if (!job.getRequestedBy().hasPermission(job.getType())) {  
    throw new RuntimeException("User not allowed for this job type");  
}
```

2. Recording the execution time:

```
long start = System.currentTimeMillis();
```

3. Take a connection from ConnectionPool:

```
Connection conn = pool.acquire();
```


4. Passing the job to the actual JobExecutor

```
realExecutor.executeJob(job);
```

5. Reconnect to the Pool

```
pool.release(conn);
```

6. Print execution time

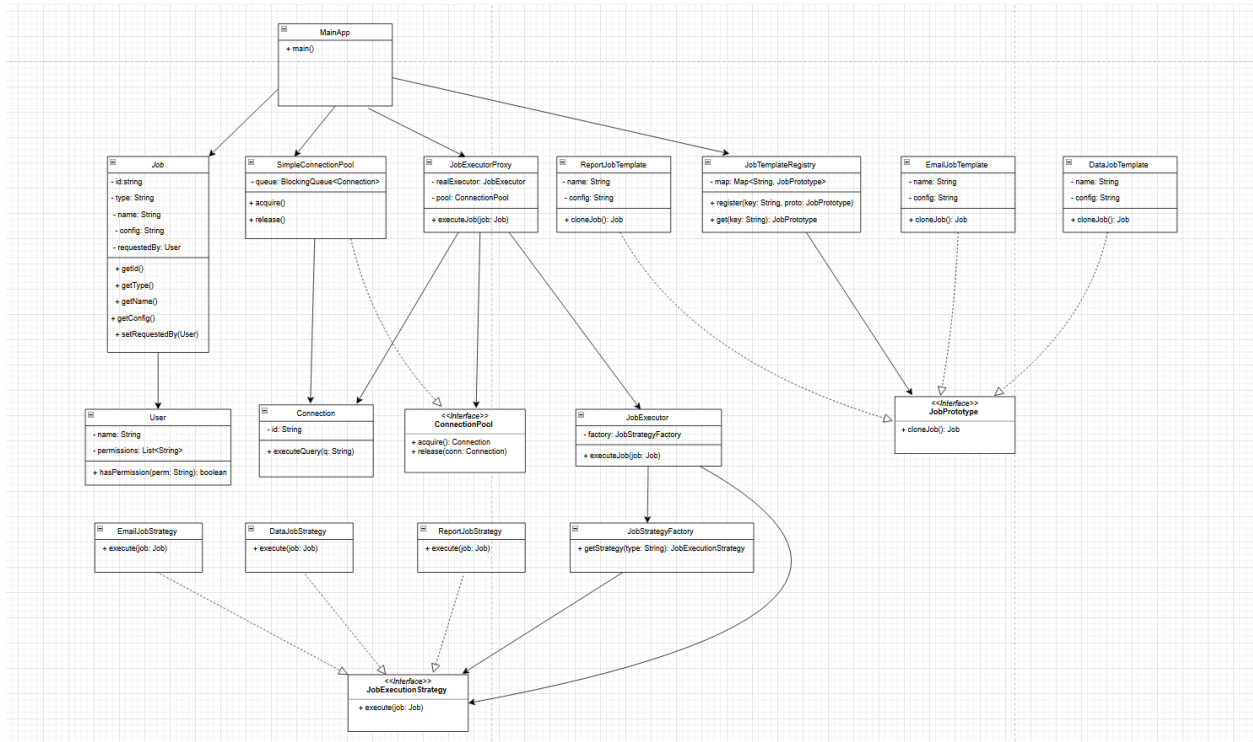
```
System.out.println("[Proxy] Execution time: " + duration + " ms");
```

The final output:

```
C:\Users\MASTAER\.jdk\openjdk-25\bin\java.exe "-javaagent:C:\Program Files\JetBrains\
[Proxy] Starting job: SalesMonthly
[ReportJob] Generating report: SalesMonthly
Config: format=PDF
[Proxy] Finished job: SalesMonthly
[Proxy] Execution time: 15 ms

Process finished with exit code 0
```

UML class diagram:



Design Decision Document

Describe:

- Why each design pattern was chosen

Connection Pool Why did we choose it?

- Because creating a new connection for each job is expensive and slow.
- The Pool allows for connection reuse and reduces resource consumption.
- An alternative like creating a connection for each process is inefficient.

Prototype Why did we choose it?

- Building templates used to take 3 seconds each time.
- The Prototype allows for quickly copying a ready-made template without rebuilding it.
- Factory alone is insufficient because it would always rebuild the template.

Strategy Why did we choose it?

- To eliminate the long if/else function within the JobExecutor.
- Each Job type now has its own independent class.
- Adding a new Job type is easier without modifying the JobExecutor.

Proxy Why did we choose it?

To add:

- User permission verification

- Logging
- Execution time measurement
- Connection management from the Pool
Without modifying the core code.

- What alternatives you considered

Connection Pool

Alternative: Creating a new connection for each job
Slow , Restrictive , Not scalable.

Prototype

Alternative: TemplateFactory builds a template every time
Always incurs the same cost , Inefficient.

Strategy

Alternative: Switch-case or if/else
Violates OCP and makes JobExecutor not scalable.

Proxy

Alternative: Placing logging + authorization within JobExecutor
Causes overlapping responsibilities and breaks SRP.

- How the patterns interact

- MainApp creates templates using the Prototype Registry.
- The proxy receives the job and then:
 - 1.Verifies user permissions.
 - 2.Obtains a connection from the Connection Pool.
 - 3.Passes the job to the JobExecutor.
 - 4.The JobExecutor uses a strategy to determine the execution logic.
- After execution:
 - The proxy records the time.
 - Returns the connection to the Pool.

The entire system utilizes all patterns in a seamless manner.

- How your architecture improves scalability, flexibility, and maintainability

Scalability

- Connection Pool allows for the execution of a large number of jobs without consuming resources.
- Prototype reduces job creation time when dealing with a large number of jobs.

Flexibility

- Strategy allows adding new job types without modifying existing code.

-Proxy allows adding new features without changing the execution logic.

Maintainability

-Segmented Responsibilities Reduction (SRP) across all components.

-Directed Delay (DRY).

-Easier unit testing for each component.

Issue Analysis Report

1) Issues in MainApp

The original code rebuilt templates from scratch each time, causing significant slowdowns and unnecessary redundancy.

It also relied directly on ConnectionManager, creating a strong dependency and making modifications difficult.

After the redesign, we used Prototype to avoid rebuilding and Connection Pool for better connection management.

2) Job Issues

The job type was stored as a single text file, which led to errors and made verification difficult.

Also, the config file was a single, unstructured piece of text, and the requestedBy file was changeable after the job was created.

After modification, the execution logic became separate using a strategy, and permission checks were performed via a proxy, not within the job itself.

3) ConnectionManager Issues

It created a new connection for each job, without a pool, which consumed resources and caused slowdowns.

It could also exceed the maximum connection limit because it didn't use proper connection management.

The solution was to create a SimpleConnectionPool that allowed for reusing connections and adjusting their number.

4) TemplateManager Issues

The code was very repetitive, and each template was created from scratch, resulting in poor performance.

Its responsibilities were numerous and unclear.

We used Prototype + Registry to provide ready-made versions of templates without duplication or high costs.

5) JobExecutor Issues

The code contained a large if/else statement to define the job type, which violates OCP principles and makes adding new types difficult.

It also handled everything: calling, executing, and printing, which violates SRP.

After the redesign, we used the Strategy Pattern to isolate the execution logic and a proxy to add permissions, time, and pool connections.

6) System Issues as a Whole

The old design was weak in terms of scalability and maintainability, and it had significant redundancy and a lack of responsibility separation.

The new design is more modular and scalable, and each part operates independently using the four Design Patterns.